PQC-LINK: Progress Report 2

Reliable delivery, verification on the wire, and preparation for Phase 2

RSCL senior design team

October 19, 2026

Abstract

Report 1 described the secure link itself: a TLS 1.3 client and server that compare classical key exchange (configuration A, `X25519`) with hybrid post-quantum key exchange (configuration B, `X25519MLKEM768`), and a first handshake benchmark. Since then the team added what the link needs to be useful: a durable store on the receiving side, a persistent queue on the sending side, tests for outages, restarts and packet loss, and the server's half of the handshake cost. We also checked what actually crosses the wire with a real packet capture and damaged encrypted records on purpose to see that they are refused. Finally we started the Phase 2 software that needs no hardware: the UART frame format and the code that turns sensor frames into records. Everything here was measured or tested on one development computer over loopback. None of it is gate evidence, and the numbers are *preliminary*.

1 Since Report 1

Report 1 ended with a working link, a failure-case test suite and a benchmark harness; the durable store, the queue and the loss tests were listed as "next" or "planned". Table 1 shows what changed, and Figure 1 shows the same on a map of the system. The automated test suite grew from 10 tests to 22.

Table 1: Status against Report 1. "Tested" means automated tests pass on the development computer.

Area	What exists now	State
Durable store, dedup	Append-only file, `fsync` before the ACK, duplicate recognised by node and sequence	Tested
Persistent queue	Append-only file, sequence numbers owned by the queue, ACK entries, atomic compaction	Tested
Client resend	Window of 32, reconnect with backoff, resend from the oldest un-ACKed record	Tested
Outage, restart, overflow	T05, T06, T07 and the queue-full case run as real processes	Tested
Packet-loss runs (T08)	Runner with `tc netem` at 0, 10, 30 %, no root needed	First look
Handshake cost (T10)	Client *and* server CPU, memory and bytes	First look
On the wire	Real packet capture; damaged records refused over real TCP (T09)	Tested
UART frame, sample source	Frame format, parser, gap counting; the Pico side shares the code	Tested
Pico firmware, IMU, OLED, Wi-Fi	Phase 2, after the advisor reviews Phase 1	Deferred

2 Decisions since Report 1

Table 2 lists the design decisions made since Report 1. Those marked *proposed* are written in the protocol document with the value we suggest; they need the advisor's confirmation.

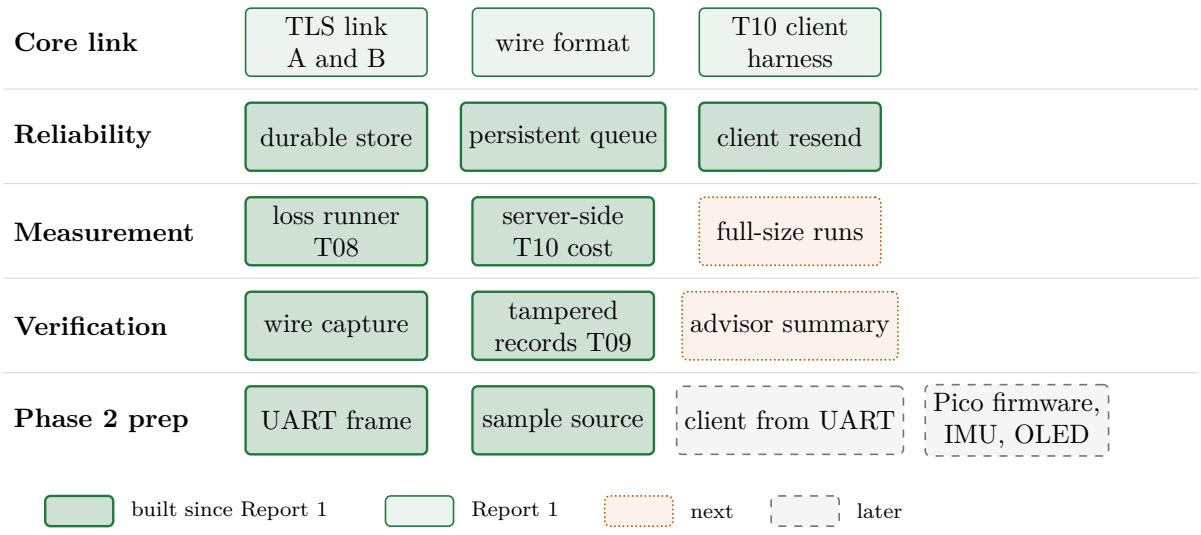


Figure 1: Progress map. Dark green: built since Report 1.

Table 2: Decisions. "Team" = decided by the team; "proposed" = written down, awaiting the advisor.

Decision	Status	Why
Durable files are append-only with a check value per entry; only a torn *last* entry is cut off, any other damage stops the program	Team	A power cut can only tear the last write; anything else would lose acknowledged data
Not SQLite: a plain file with one `fsync` per record	Team	No new dependency, every durability step is short and tested
The queue owns the sequence numbers; a header floor stops a number being reused after compaction	Team	A reused number would be treated as a duplicate and lose data
Queue holds 10,000 un-ACKed records; a full queue rejects, counts, and still uses up the number	Proposed	Never drop a record already promised; the gap shows the loss
A handshake that times out is retried; an alert, a failed verification or a policy mismatch is never retried	Team	Packet loss must not end a run, but a retry loop must not hide T03 or T04
Pico 2 or Pico W does sensor and display I/O only; Pico firmware is C with the Pico SDK	Team, confirm	They cannot run TLS; all TLS stays on the Pi 4B
Code shared with the firmware may use only the standard integer types and the error codes	Team	Both ends then compile the same file and cannot disagree about the format

3 Reliable delivery

3.1 The path of one record

The rule behind the design is: *a side may forget a record only after the other side holds it durably.* Figure 2 shows it. The sender writes a record to its queue file and forces it to disk before sending; the receiver writes it to its store and forces it to disk before acknowledging; the sender removes the record only after the acknowledgement arrives. If an acknowledgement is lost, the sender sends the record again and the receiver recognises it by its node and sequence number, acknowledges again, and changes nothing else. The count shown on the display therefore cannot go up twice for one record.

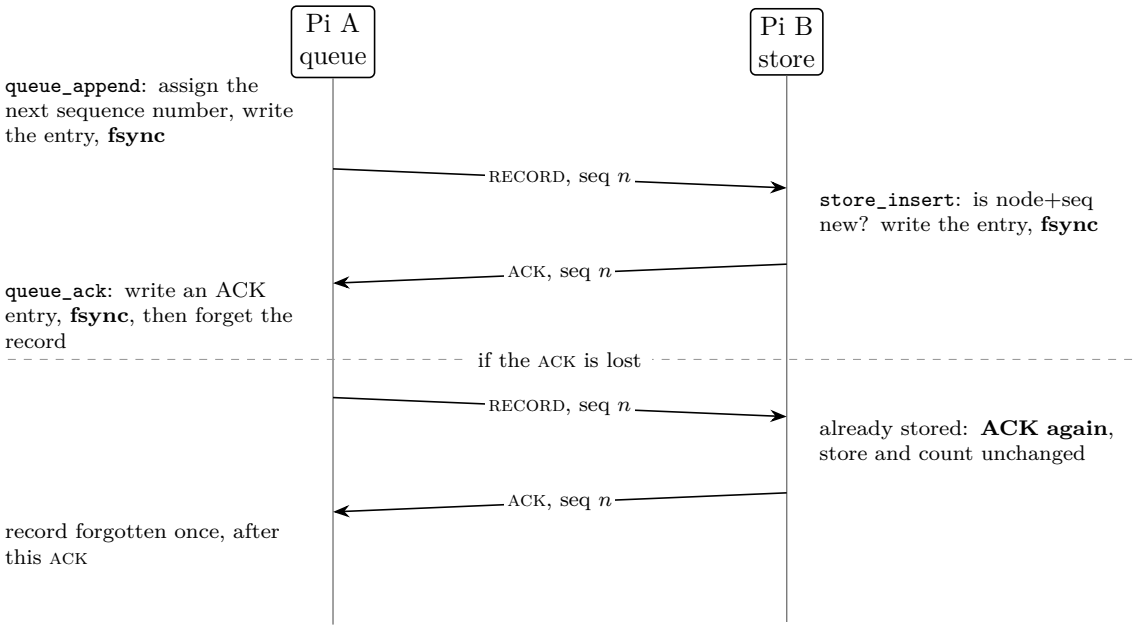


Figure 2: One record, with the two points where data is forced to disk.

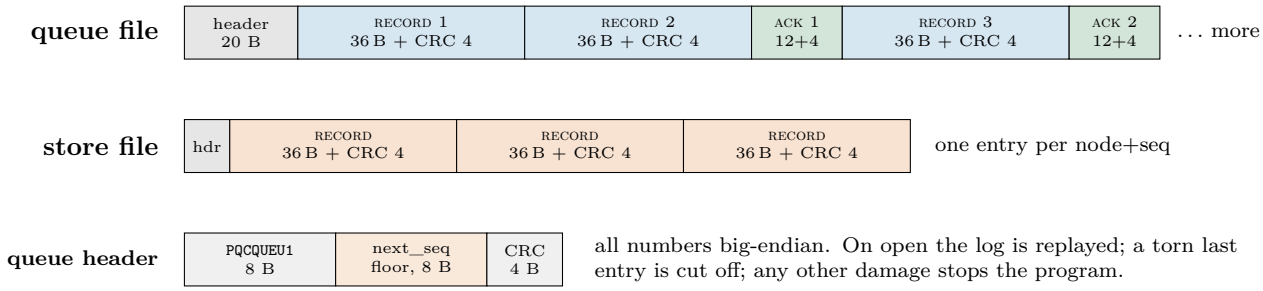


Figure 3: Queue and store files. A record is waiting in the queue if no ACK entry follows it.

3.2 On disk

Both files are logs that are only added to. Figure 3 shows their layout. An entry is a message exactly as it travels on the network plus a four-byte check value (CRC-32). The queue needs a second kind of entry, an ACK, and a small header holding a *floor* for the next sequence number: compaction (writing a new file with only the waiting records and renaming it over the old one) drops the entries that mention old numbers, and without the floor a restart could hand the same number out again.

3.3 What a crash does

Table 3 lists the failures the tests inject and what the code does. The queue was also checked against a model: 3000 random appends, ACKs (in order, repeated and out of order) and restarts compared with a plain array, and no sequence number is ever reused.

Table 3: Crash and damage cases, all covered by automated tests.

What happens	Result
Power cut in the middle of writing the last entry	Entry cut off at start; it was never acknowledged, so nothing is lost
Server stores a record, ACK never arrives	Client resends; server acknowledges again; counted once (T05)
Client stopped (or killed with SIGKILL) with records waiting	On restart the waiting records are sent first; numbering carries on (T07)
Server down for the whole run	Queue grows, then drains after reconnect (T06)
Queue full	New records rejected and counted; the gap in sequence numbers shows the loss
Damage anywhere before the last entry, a repeated number, a file that is not a queue	Program refuses to start; the file is left untouched
Crash during compaction	Old file or new file, whole, never a mix

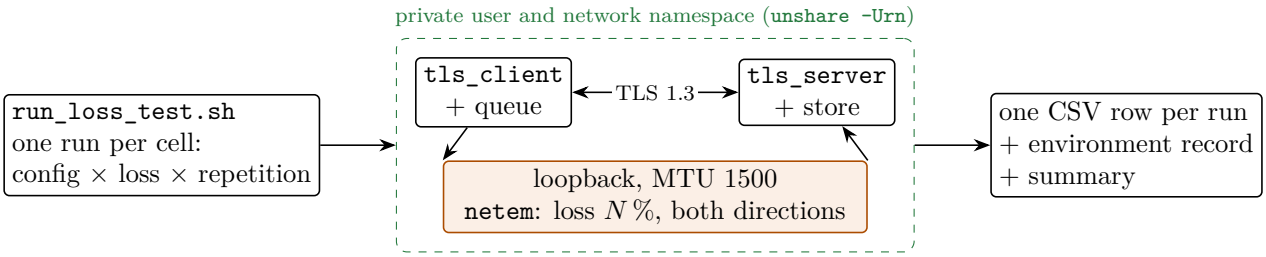


Figure 4: The loss runner. A and B alternate within each loss level so drift hits both equally.

4 Packet loss (T08)

4.1 Method

Loss is emulated with `tc netem` on the loopback interface. Doing that normally needs root. The runner (Figure 4) instead starts the server and the client inside a fresh, unprivileged user and network namespace, where the team has its own loopback and the right to change it. Runs cannot disturb each other or the machine, and the namespace's TCP counters hold exactly one run. The loopback is set to the Ethernet MTU of 1500 bytes, so a long message is split into packets as it would be on Wi-Fi. Loss applies to every packet in both directions, so a request and its answer can each be lost.

A run counts as *delivered* only if nothing is still waiting in the queue, the number of records the server stored equals the number the client accepted, and the client's count of ACKs agrees. The server's count is read from the size of its store file, not from its log. Every run is reported, delivered or not; a result with fewer than 10 runs per cell is marked insufficient.

4.2 First look

Table 4 and Figure 5 show one development-sized matrix: 3 repetitions per cell, 200 records per run at 20 Hz, 18 runs in total, measured on 2026-10-20. This is far below the 10 repetitions of 10 minutes that the specification asks for, so **none of these numbers is a result**; they show that the runner works and what to expect. Out of 18 runs, 18 delivered every accepted record and 0 records were rejected by a full queue.

Table 4: T08, development-sized. Medians over the repetitions of each cell. Drain time is the time from the last record made until nothing is waiting. Retransmissions are TCP segments, read from the namespace's own counters: they show the emulated loss acted.

Config	Loss	Runs	Delivered	Lost sessions	TCP retrans.	Drain, ms	Drain p95, ms	Run, s
A	0 %	3	3	0	0	6	10	10.0
B	0 %	3	3	0	0	6	8	10.0
A	10 %	3	3	0	51	14	80	10.0
B	10 %	3	3	0	43	16	21	10.0
A	30 %	3	3	3	76	5 114	53 151	15.8
B	30 %	3	3	1	70	3 591	19 371	14.2

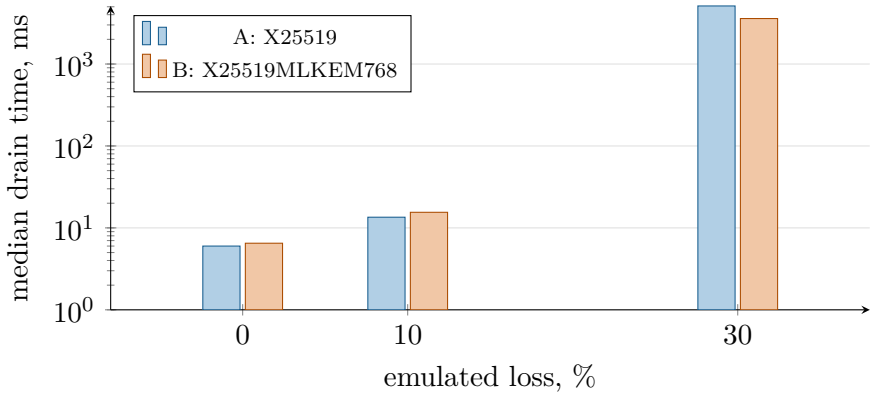


Figure 5: Median time to empty the queue after the last record, by emulated loss. The vertical axis is logarithmic: at 0 % and 10 % the queue empties in milliseconds, at 30 % in seconds.

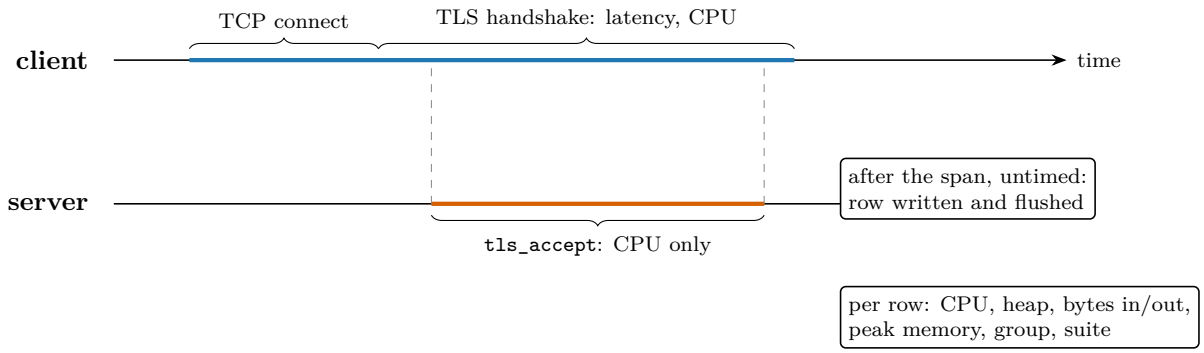


Figure 6: What each side measures. Neither writes anything during its timed span.

5 The cost of a handshake, on both ends

Report 1 measured only the client. The server now writes one row per accepted connection (Figure 6): the CPU time of the server process over the handshake call itself (waiting for the client is not counted), the heap the open connection holds, the handshake bytes, and the process's peak memory. Rows are written after the timed span, never inside it. The summary drops failed and warm-up rows and refuses to continue if the server's count differs from the client's.

5.1 Conditions

- Platform: Intel(R) Core(TM) Ultra 9 285H, WSL2; client and both servers on one machine over loopback, Release build.
- 3 sessions, 300 measured handshakes per configuration in total; run date 2026-10-20.

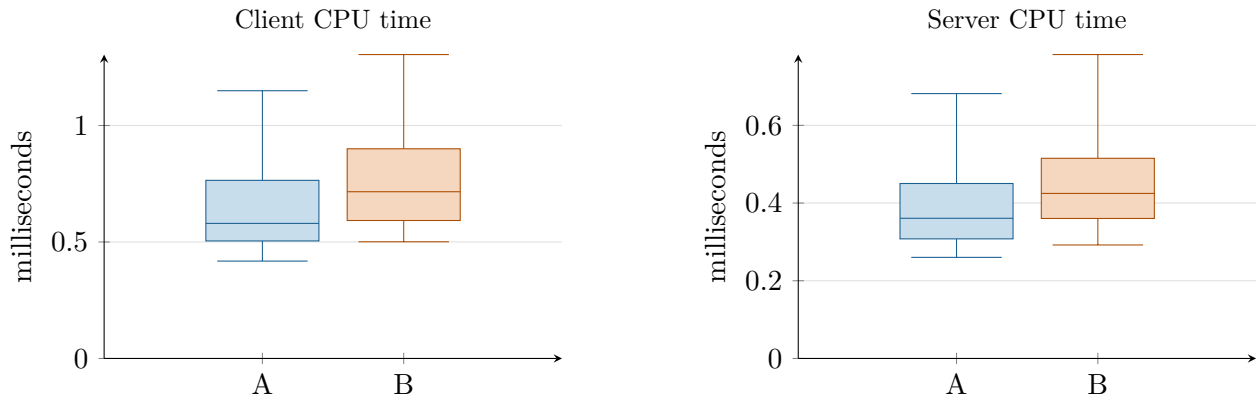


Figure 7: CPU time of one handshake. Box: 25th to 75th percentile with the median line; whiskers: 5th and 95th percentile. Blue = A, orange = B.

- Both sides counted the same bytes in every handshake (client sent = server received and the reverse): yes.

Preliminary: one development computer, loopback, no loss.

5.2 First look

Table 5 puts the two sides next to each other. In this run the hybrid handshake cost the server about 1.18 times the CPU time of the classical one, and the client about 1.23 times. The open connection holds more memory in B on both sides. The server's peak memory is the whole process (8496 and 8488 kB), dominated by its baseline, so it does not separate A from B; the heap held by the connection does.

Table 5: Handshake cost, client and server, medians (95th percentile in parentheses).

	A: X25519		B: X25519MLKEM768	
	client	server	client	server
CPU time, ms	0.580 (1.149)	0.361 (0.682)	0.716 (1.304)	0.425 (0.782)
Heap held by the connection, KiB	19.1	15.2	26.8	21.1
Handshake bytes sent	273	755	1449	1843
Handshake bytes received	755	273	1843	1449

6 What crosses the wire

Phase 1 has to show verified encryption and decryption, and a unit test on a socket pair is a weak form of that. We therefore put a small TLS-aware proxy between the real client and server, and in the same private namespace recorded the loopback interface to a real `.pcap` file (Figure 8). The proxy reads the one part of TLS 1.3 that is not encrypted, the two hello messages, and can flip a single bit in a chosen encrypted record. The packet capture gives the headers; the proxy gives an independent count of the same bytes.

6.1 What the capture shows

In both configurations the first message from the client names exactly one key-exchange group, the pinned one (`001d` for A, `11ec` for B), offers TLS 1.3 only and one cipher suite (`1301`). The server

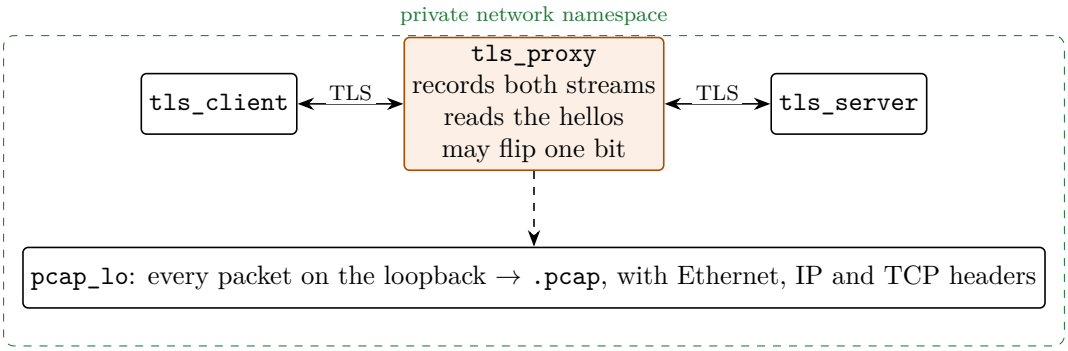


Figure 8: The test bench. The proxy sees the TLS stream; the capture sees the packets.

chooses the same group (`001d`, `11ec`) with no retry, so nothing was negotiated down. After the two hello messages every record is encrypted: the server's certificate cannot be read from the wire. None of the five queued records, none of their sequence-number and time fields, and none of the server's ACKs appears anywhere in the captured bytes, while the same records *are* in the server's store, which shows the search could have found them. The captured payload equals the proxy's own byte count in both directions (A: yes; B: yes), so the two captures agree and no packet was counted twice. Table 6 gives the byte totals with headers for a run of five records.

Table 6: One run, five records, on the wire. "On the wire" is the whole Ethernet frame. Headers add 1456 bytes over 22 packets in A and 1456 over 22 in B; the clean loopback had 0 retransmissions in A and 0 in B.

Config	Direction	Packets	TLS payload, B	On the wire, B
A	client → server	15	587	1573
	server → client	7	950	1420
B	client → server	15	1763	2749
	server → client	7	2038	2508

6.2 Damaged records are refused (T09)

Table 7 shows the four places where the proxy flipped one bit. In every case TLS refused the damaged record, nothing damaged was stored, and the record still arrived exactly once. Figure 9 shows the first case. A damaged ACK followed by the resend exercises two requirements together: the client must not trust the ACK, and the server must recognise the duplicate (T05 and T09).

Table 7: Where one bit was flipped, configuration B, and what happened.

Damaged record	Result
Client's RECORD	Server rejects it (authentication tag), stores nothing, sends no ACK; client reconnects and delivers; stored once, never the damaged bytes
Server's ACK	Client refuses it and reconnects; resends; server recognises the duplicate, ACKs again; stored once
Server's first encrypted handshake record	Client's handshake fails and the client stops: it fails closed and does not retry
Client's Finished	Server refuses the connection; the client notices when no ACK comes, reconnects and delivers

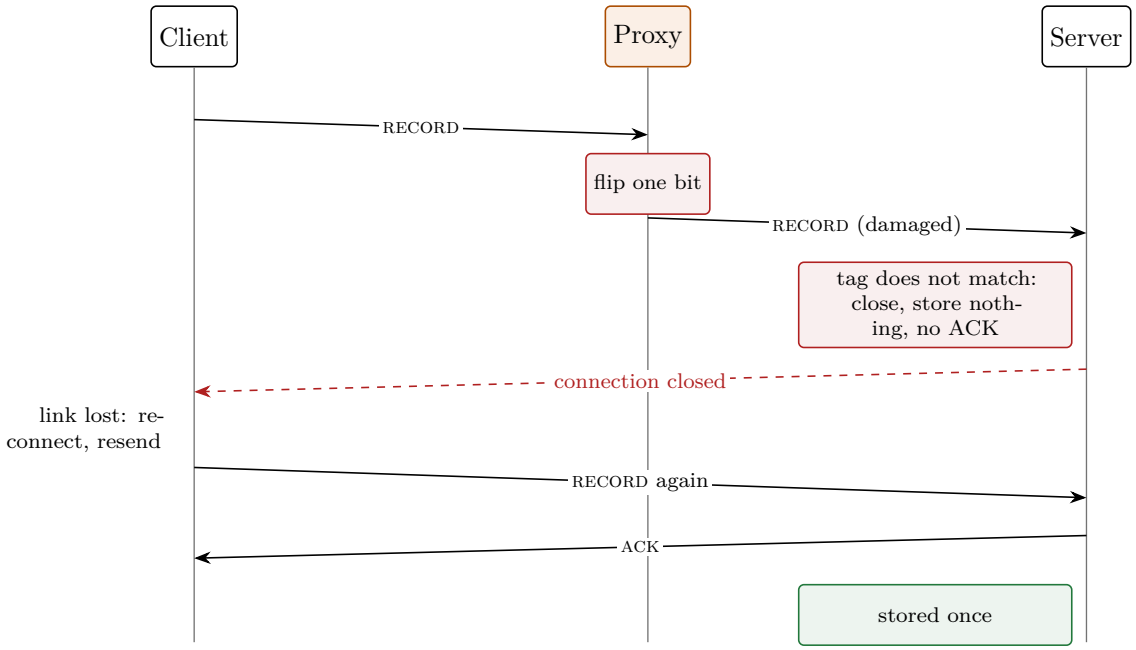


Figure 9: A damaged RECORD. The proxy damages the first connection only.

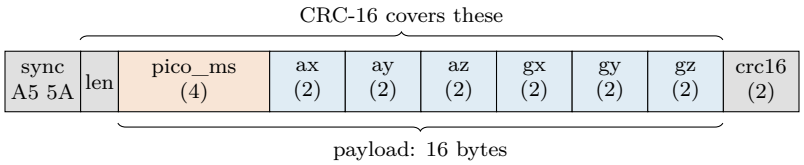


Figure 10: One sample on the UART, 21 bytes, big-endian. Widths are proportional to bytes.

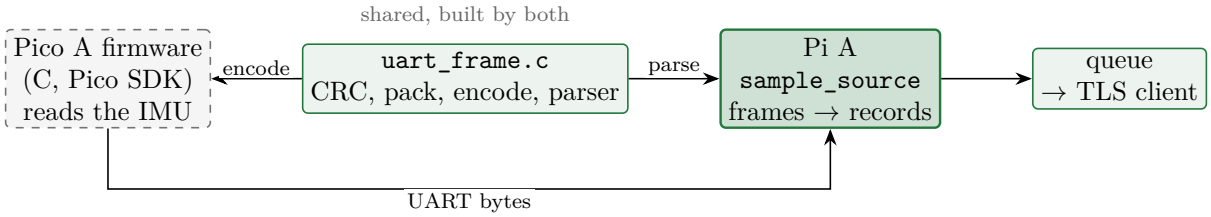


Figure 11: One file for both ends. The firmware part (dashed) is Phase 2.

7 Preparing Phase 2 without hardware

The Pico side is not built, but its software contract can be fixed and tested now. The UART frame between Pico A and Pi A (Figure 10) is 21 bytes for one sample. A single C file builds and parses it, and *both* the Pico firmware (with the Pico SDK) and Pi A compile that same file (Figure 11), so the two ends cannot disagree about the format.

7.1 Surviving a damaged cable

The parser keeps a window of at most 37 bytes (the longest frame). When a candidate frame fails its check, it drops only the *first* byte and looks again, so the bytes behind a damaged frame are searched for the next real frame (Figure 12). A lost byte, an extra byte or a false sync pattern therefore costs only the damaged frame. A parser that discards the whole claimed frame would also lose the good frames it swallowed. Tests cover every possible single-bit flip of a frame, every possible lost byte, inserted bytes at every position, random noise, and a 300,000-byte random stream.

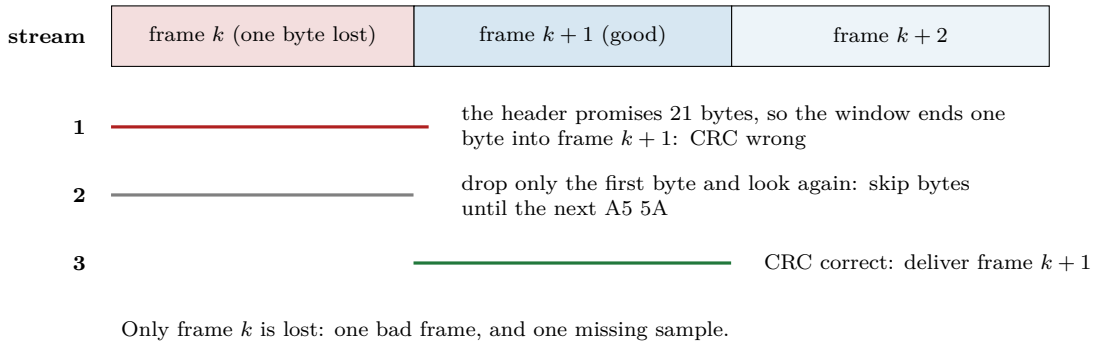


Figure 12: Re-synchronisation after a lost byte.

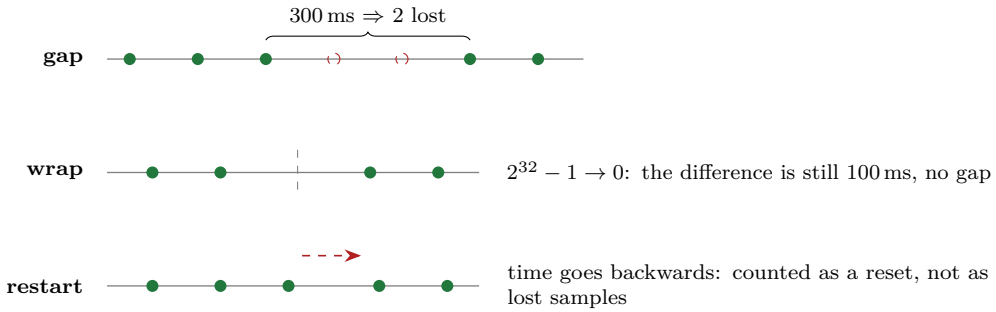


Figure 13: Gap rules. Filled: received. Dashed: never arrived.

7.2 Counting missing samples

Pi A turns each good frame into the start of a record (the queue adds the sequence number) and counts what went wrong. Missing samples are found from the Pico's own timestamp `pico_ms`, with unsigned 32-bit subtraction so the wrap at 2^{32} is not a gap (Figure 13). A jump of ten seconds or more, or time going backwards, is a restart of the Pico, not millions of lost samples. In a test of 1200 frames with some dropped and some damaged, the count of lost samples equalled the number dropped plus the number damaged exactly. One caution: the record's timestamp `ts_ms` is when Pi A *read* the frame, not when the Pico sampled, because the driver holds bytes until they are read.

8 How the work is checked

Every module has a plain C test program and the end-to-end scripts start the real client and server as separate processes. Besides the happy path, each new feature has an adversarial test: wrong length, damaged input, a partial write, an existing file that must not be overwritten, a process that must not hang. Table 8 groups what the 22 automated tests cover. We also break the code on purpose (a *mutation*: an off-by-one, a check removed, a wrong byte order) and confirm that a test fails; this caught several tests that passed for the wrong reason. Code is built with all warnings as errors, with address and undefined-behaviour sanitizers, and tests that use timing or pseudo-terminals are repeated 40 times to rule out flakes.

9 What remains to close Phase 1

Figure 14 shows the road. The software is essentially complete. What remains is running the measurements at full size, writing them up, and the advisor's review, which is also the gate to Phase 2.

Table 8: Test areas.

Area	Covers
Wire format, UART frame	Byte vectors, every bit flip, lost and extra bytes, noise, false sync
Store, queue	File layout, restart, torn last entry, damage, full, ACKs in any order, compaction, a model test
TLS link, client, server	A and B, downgrade, wrong hostname, untrusted CA, tampered record, timeout, replay, outage, restart
Loss and cost runners	Delivery verdicts, percentiles, server and client byte agreement, netem really acting
Wire and capture	Pinned group on the wire, no plaintext, damaged records refused, capture equals proxy count
Sample source	Raw serial mode (all 256 byte values), gap rules, 1200-frame accounting, port timeout and disconnect

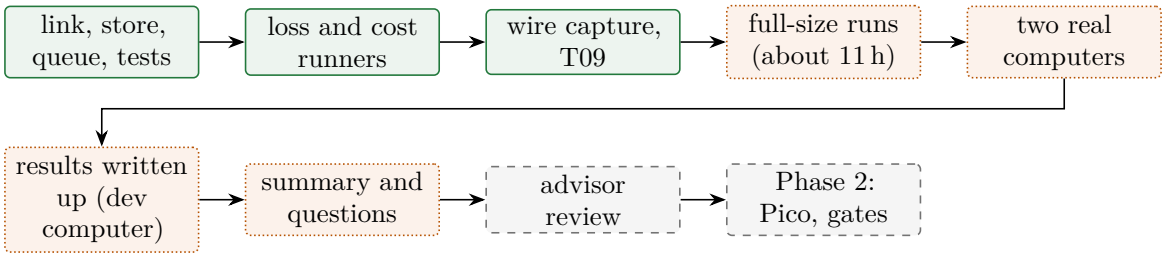


Figure 14: Remaining steps. Solid: done. Dotted: next. Dashed: waits for the advisor.

9.1 Questions for the advisor

- **Pico role.** We understand the Pico 2 or Pico W does only sensor and display I/O and all TLS stays on the Pi 4B, because the Pico cannot run TLS. The advisor's wording of "migration to Pico W" could be read as a platform change; please confirm.
- Q5: which instrument for current and power, and is per-handshake energy needed? This decides whether a sensor is bought.
- Q10: is packet loss emulated with `tc netem` on the Pis or at the access point?
- Q11: may the Pi services stay in C?
- Q6 and Q8: confirm there is no rekey and no command channel.
- Q15: should the existing logger be reused for the evidence files?
- Confirm the queue size of 10,000 records and the reject-new rule.

10 Claims and limits

Hybrid key establishment protects the *key exchange* against a future quantum computer. It is **not** post-quantum authentication: the baseline authenticates with an ordinary classical certificate, and this report says so wherever it matters. This is an educational Wi-Fi link, not flight-approved, not space-qualified and not endorsed by any agency. All measurements here come from one development computer over loopback; they show that the harnesses work and give a first idea of the sizes and costs. They must not be quoted as results for the Raspberry Pi 4B or the Pico, and they are not gate evidence.

A Reproducing the numbers

Every number in Sections 4 to 6 is generated from raw files by `prepare_data2.py`; nothing is typed.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DOPENSSL_ROOT_DIR=$HOME/opt/openssl-3.5.9
cmake --build build && ctest --test-dir build
scripts/run_handshake_bench.sh $HOME/opt/openssl-3.5.9 build \
    experiments/configs/t10_handshake_loopback.conf results/
scripts/run_loss_test.sh $HOME/opt/openssl-3.5.9 build \
    experiments/configs/t08_loss_loopback.conf results/
scripts/capture_demo.sh $HOME/opt/openssl-3.5.9 build results/wire_demo
python3 -I docs/report/prepare_data2.py results results/wire_demo \
    docs/report/data2 build
cd docs/report && latexmk -pdf progress_report_2.tex
```